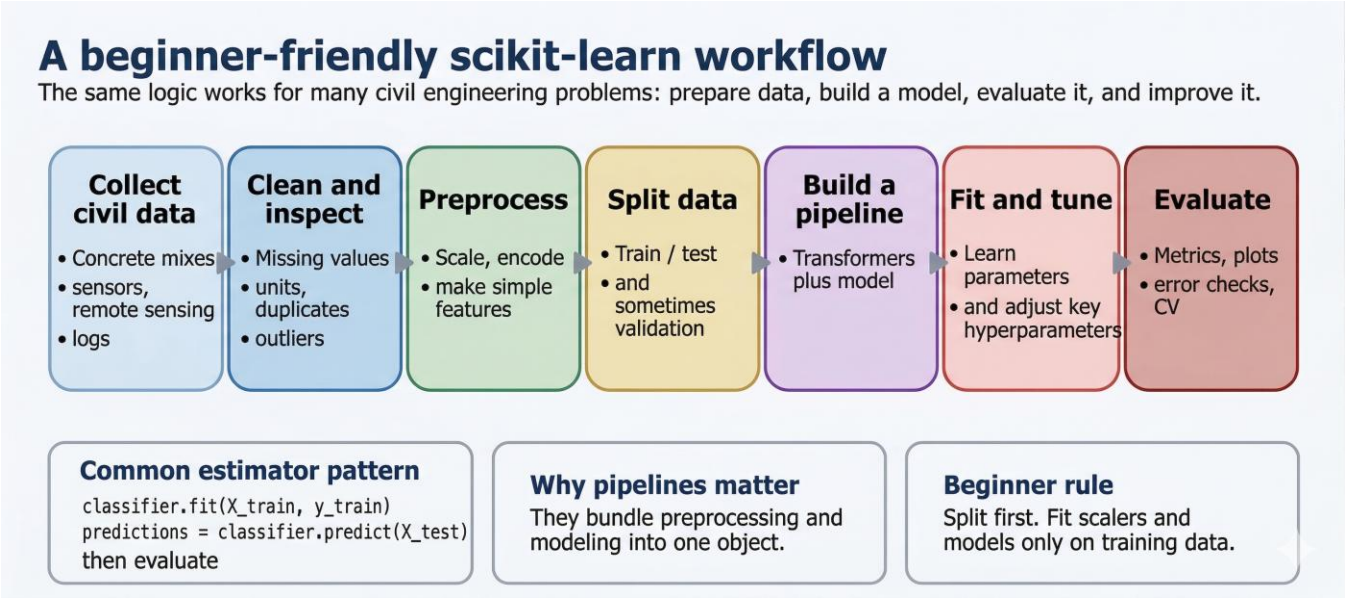
# Practical scikit-learn for Civil Engineering

A beginner-friendly tutorial on data preparation, classical machine learning models, evaluation, and reliable workflows in Jupyter Notebook or Google Colab

**What this tutorial covers**

Cleaning, scaling, calibration, train/test splitting, preprocessing for tabular, image-derived, and time-series data, plus support vector machines, k-nearest neighbors, decision trees, metrics, plots, pipelines, and cross-validation.

Audience: Civil Engineering students with beginner Python skills. The examples stay simple, use plain Python syntax, and are designed to be copied into notebook cells with minimal editing.




*This tutorial follows the stable scikit-learn documentation available at the time of writing (version 1.8.0) and uses the UCI Air Quality, Concrete Compressive Strength, and Statlog (Landsat Satellite) datasets as teaching examples [1]-[17].*

# How to use this tutorial

Read the short explanations first, then run the code cells section by section. Each code block is written in a notebook style. Lines beginning with ! run shell commands in Jupyter Notebook or Google Colab.

### A good beginner habit

Do not start with the most advanced model. First clean the data, inspect the columns, choose a simple train/test split, fit one baseline model, and only then compare alternatives.

*Table 1. Plain-language glossary for the main scikit-learn ideas used in this tutorial.*

| Term | Plain-language meaning | Typical example |
|---|---|---|
| sample or data point | One row or one case that the model learns from. | one concrete mix, one sensor record, one image patch |
| feature | An input variable used to make a prediction. | water content, age, or a spectral value |
| target | The output the model tries to predict. | compressive strength or land-cover class |
| estimator | A scikit-learn object that can learn from data. | SVC, KNeighborsClassifier, DecisionTreeRegressor |
| transformer | An object that changes the data before modeling. | StandardScaler or MinMaxScaler |
| parameter | A value learned from the training data. | tree split values or SVM support vectors |
| hyperparameter | A setting chosen by the user before training. | k in KNN, max_depth in a tree, C in SVM |
| pipeline | A chain that keeps preprocessing and modeling together. | scale -> model |
| metric | A number used to judge model quality. | MAE, $R^2$, accuracy, precision, recall |

# 1. What scikit-learn does for you

scikit-learn is an open-source Python library that supports supervised and unsupervised learning, and also provides tools for preprocessing, model fitting, model selection, and model evaluation [1], [2].

For beginners, its biggest advantage is consistency. Most objects follow the same basic pattern: prepare X and y, split the data, call fit on the training set, call predict on new data, and then evaluate the result with one or more metrics [1], [2].

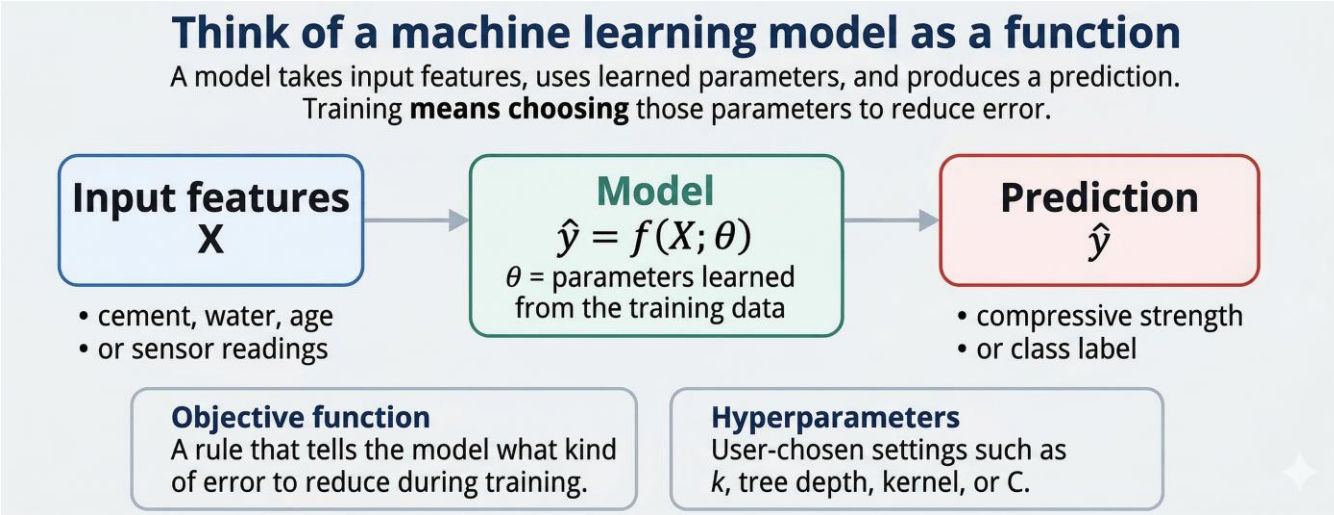



*Figure 1. A machine learning model can be viewed as a function that maps input features X to a prediction ŷ using learned parameters. Training reduces an objective function while the user selects hyperparameters such as tree depth, k, or SVM C. Author-generated illustration based on scikit-learn estimator concepts [1], [2].*

**Common estimator pattern**

In simple form: model.fit(X_train, y_train), then y_pred = model.predict(X_test). Transformers use fit and transform, and pipelines let you combine both patterns in one object [3].

### Notebook setup cell

```
!pip install -q scikit-learn pandas matplotlib xlrd openpyxl

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

The import cell above is enough for most of the early examples. Later sections add only the model class or metric that is needed for the current task.

## 2. Prepare the data before you model

Real civil engineering data rarely arrive in a clean, ready-to-model form. Common jobs include removing inconsistent entries, marking missing values correctly, checking units, scaling numeric features, calibrating raw sensors, and splitting data into training and testing subsets [3]-[6].

*Table 2. Common early preprocessing jobs and why they matter.*

| Preprocessing job | Why it matters | Simple scikit-learn or pandas tool |
|---|---|---|
| clean missing values | Prevents hidden errors and broken plots. | replace, isna, dropna, fillna |
| inspect columns and units | Stops unit mistakes before modeling. | df.columns, describe, hist |
| scale numeric features | Keeps one large-range feature from dominating distance-based models. | StandardScaler, MinMaxScaler |
| separate X and y | Makes the modeling step clear and reusable. | iloc, drop, last column as target |
| split before fitting | Reduces leakage and gives an honest test score. | train_test_split, Pipeline |

## 2.1 Clean inconsistent values and mark missing values correctly

The UCI Air Quality dataset contains hourly sensor responses together with reference pollutant measurements. Its missing values are tagged with the number -200, so the first job is to replace that code with NaN before analysis [12].

A second beginner-friendly habit is to inspect the table before modeling. Check column names, count missing values, and make one or two quick plots so you understand what the data look like before a model sees them.

### Example cell: basic cleaning for the Air Quality dataset

```
!wget -q \
https://archive.ics.uci.edu/ml/machine-learning-databases/00360/AirQualityUCI.zip
!unzip -o -q AirQualityUCI.zip

air = pd.read_csv("AirQualityUCI.csv", sep=";", decimal=",")
air = air.dropna(axis=1, how="all")
air = air.replace(-200, np.nan)

print(air.head())
print(air.isna().sum())

air["DateTime"] = pd.to_datetime(
    air["Date"] + " " + air["Time"],
    dayfirst=True,
    errors="coerce"
)

air["CO(GT)"].plot(figsize=(8, 3), title="Reference CO over time")
plt.show()
```

### Beginner tip

Do not call dropna() on the full table until you understand how much data you will lose. First, inspect the missing-value counts. Then decide whether to drop rows, fill values, or work with a smaller set of columns.

## 2.2 Scale numerical features when ranges are very different

StandardScaler standardizes a feature by removing the mean and scaling to unit variance, while MinMaxScaler maps each feature to a chosen range, often 0 to 1 [15], [16]. Scaling is especially important for distance-based methods such as KNN and for many support vector machine workflows [4], [7], [8].

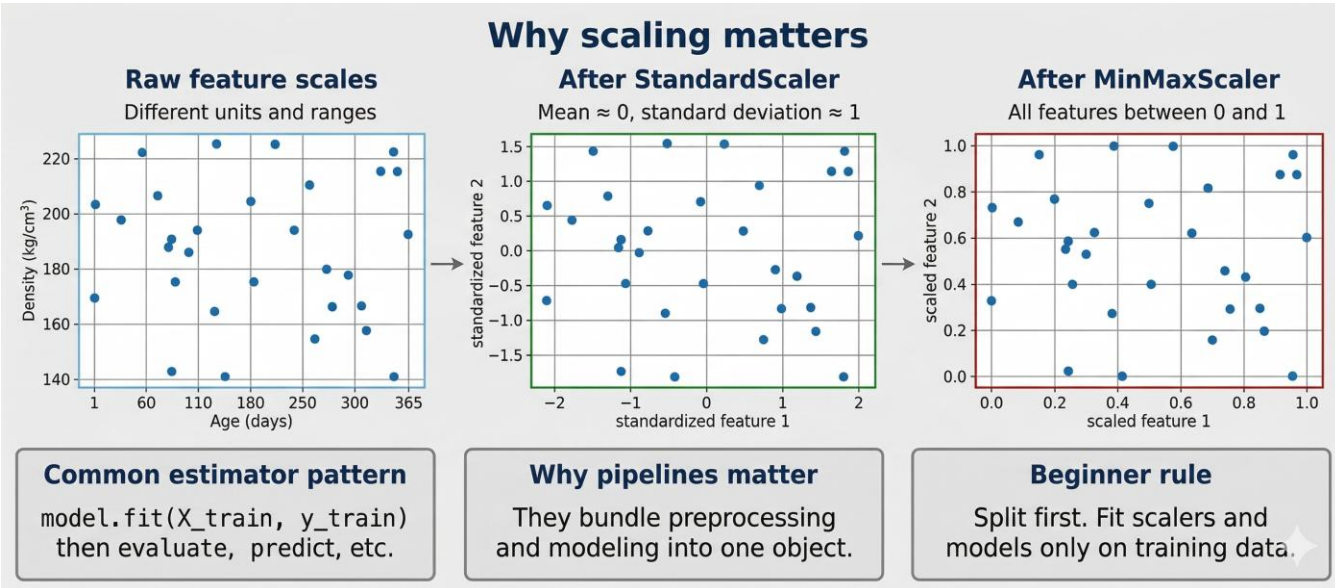



*Figure 2. Raw features can have very different ranges. StandardScaler centers and scales, while MinMaxScaler compresses each feature to a common range. Author-generated illustration aligned with the current scikit-learn preprocessing guidance [4], [15], [16].*

*Table 3. A beginner-friendly way to choose between common scaling options.*

| Tool | What it does | When a beginner should try it first |
|---|---|---|
| StandardScaler | Makes each feature roughly mean 0 and standard deviation 1. | When you use SVM, KNN, or another method sensitive to feature scale. |
| MinMaxScaler | Maps each feature to a fixed range such as 0 to 1. | When you want bounded inputs or a very simple visual scaling rule. |
| No scaling | Keeps the original units unchanged. | When the model is tree-based and units are already meaningful and consistent. |

### Example cell: scaling the Concrete Compressive Strength inputs

```
!wget -q \
https://archive.ics.uci.edu/ml/machine-learning-databases/concrete/compressive/Concrete_Data.xls

concrete = pd.read_excel("Concrete_Data.xls", engine="xlrd")

X = concrete.iloc[:, :-1]
y = concrete.iloc[:, -1]

std_scaler = StandardScaler()
```

```
mm_scaler = MinMaxScaler()

X_std = pd.DataFrame(std_scaler.fit_transform(X), columns=X.columns)
X_mm = pd.DataFrame(mm_scaler.fit_transform(X), columns=X.columns)

print(X_std.head())
print(X_mm.head())
```

The concrete dataset has 1,030 observations, 8 numeric input variables, and one numeric target, with no missing values reported by UCI [13]. Because the columns use different units and ranges, it is a good teaching example for scaling.

## 2.3 Use calibration when raw measurements need correction

Calibration means learning a correction from raw sensor output to a trusted reference value. This matters in civil engineering whenever field sensors drift, respond nonlinearly, or differ from certified instruments. In scikit-learn, a first calibration model can be as simple as a LinearRegression fit between one sensor channel and its paired reference measurement [12].

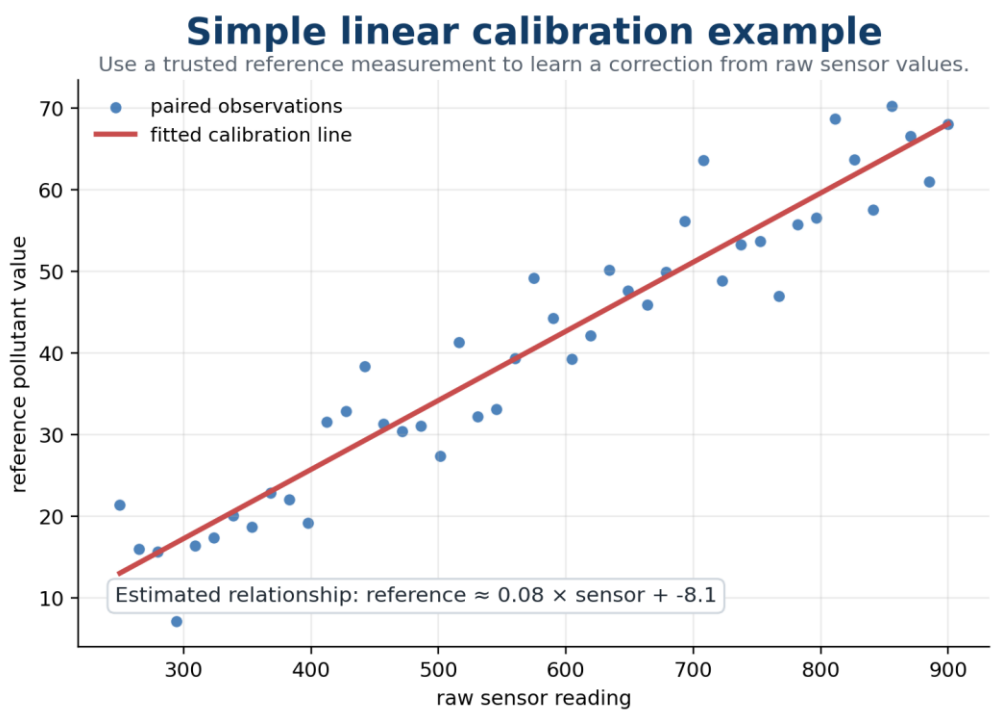



*Figure 3. A simple calibration example: fit a line that maps raw sensor readings to a trusted reference measurement. Author-generated illustration inspired by the Air Quality dataset structure [12].*

### Example cell: a simple calibration model

```
from sklearn.linear_model import LinearRegression

cal = air[["PT08.S1(CO)", "CO(GT)"]].dropna()

X_cal = cal[["PT08.S1(CO)"]]
y_cal = cal["CO(GT)"]

cal_model = LinearRegression()
cal_model.fit(X_cal, y_cal)
```

```
print("slope:", cal_model.coef_[0])
print("intercept:", cal_model.intercept_)
```

## 2.4 Split the data before fitting scalers or models

The train_test_split helper divides arrays or data frames into training and testing parts. If test_size is not given, scikit-learn uses 0.25 by default. Setting random_state makes the split reproducible [5].

For classification, use stratify=y when you want the class balance in the training and test sets to stay similar. This is a very good default when one class is rare or expensive to miss [5].

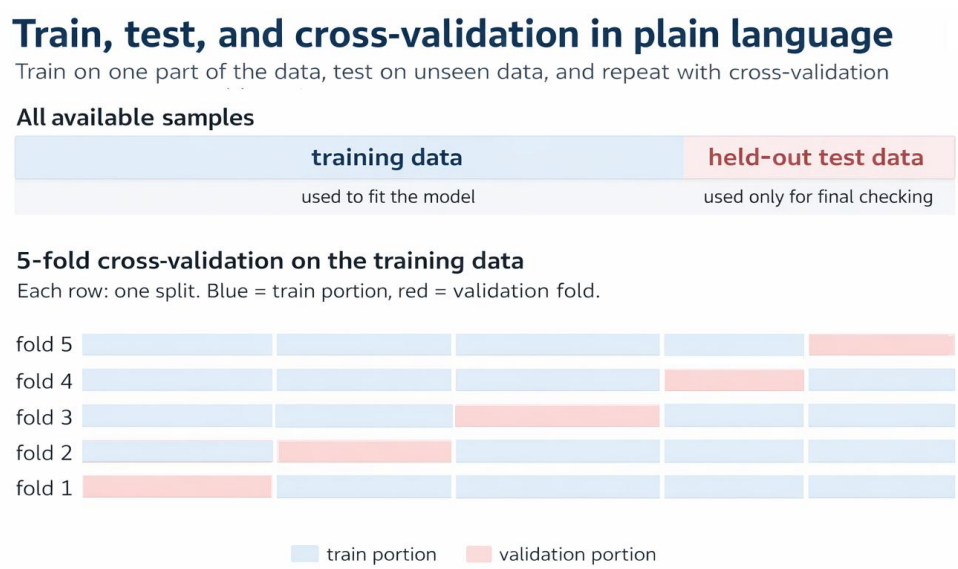



*Figure 4. Hold out a test set for the final check. Use cross-validation on the training portion when you want a more stable estimate of model quality [5], [11].*

### Example cell: regression and classification splits from the same table

```
# regression split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# simple binary classification target:
# 1 = higher-than-median strength, 0 = lower-than-median strength
y_class = (y >= y.median()).astype(int)

X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X, y_class, test_size=0.20, random_state=42, stratify=y_class
)

print(X_train.shape, X_test.shape)
print(y_train_c.value_counts())
```

# 3. Preprocessing depends on the data type

Scikit-learn works most naturally with a 2D feature matrix, but the route into that matrix depends on the original data type. Tabular data already look like rows and columns. Image-derived data often need flattening or feature extraction. Time-series data must keep their order in time [2], [4], [14], [17].

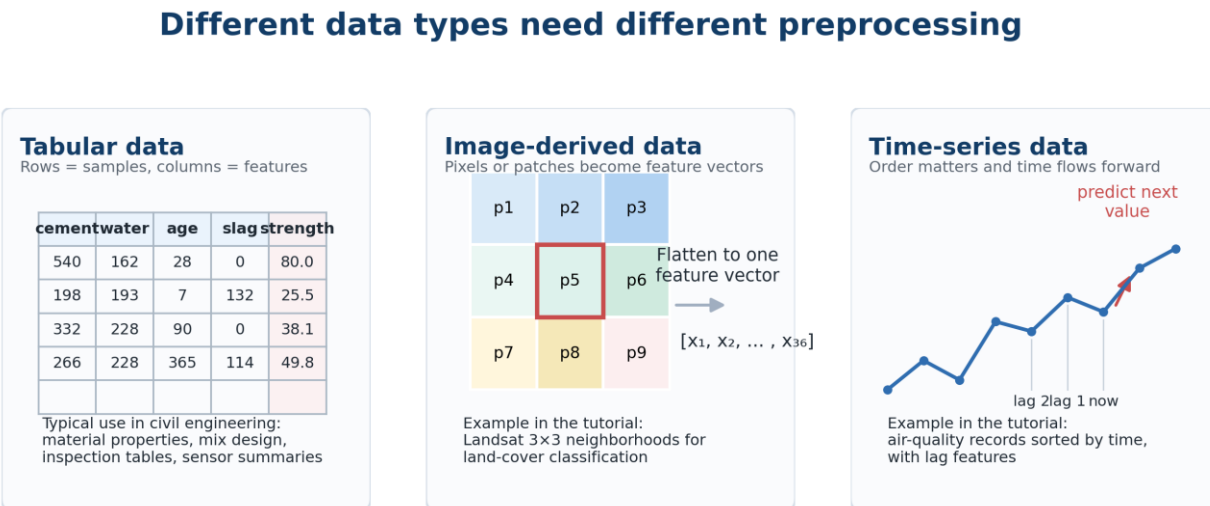



*Figure 5. Tabular, image-derived, and time-series data all enter machine learning as features, but they need different preparation steps. Author-generated illustration based on the UCI Concrete, Air Quality, and Statlog (Landsat Satellite) dataset structures [12]-[14].*

*Table 4. A quick way to think about preprocessing by data type.*

| Data type | What one sample looks like | Useful preprocessing steps | Civil example |
|---|---|---|---|
| tabular | one row of measured variables | inspect columns, handle missing values, scale numeric features | concrete mix ingredients and strength |
| image-derived | one pixel neighborhood or one image feature vector | load numeric arrays, separate labels, often scale values | remote-sensing site or land-cover classification |
| time-series | one record with a timestamp | sort by time, build lag features, split chronologically | sensor streams or air-quality trends |

## 3.1 Image-derived data can still become a normal feature table

The Statlog (Landsat Satellite) dataset is a good beginner example because it comes from imagery but is stored as rows of numbers. Each sample contains the spectral values of a 3x3 pixel neighborhood, which gives 36 input features and one label for the central pixel [14].

### Example cell: loading and scaling Landsat image-derived data

```
!wget -q \
https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/satimage/sat.trn
!wget -q \
https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/satimage/sat.tst
```

```
sat_train = pd.read_csv("sat.trn", sep=r"\s+", header=None)
sat_test = pd.read_csv("sat.tst", sep=r"\s+", header=None)

X_train_img = sat_train.iloc[:, :-1]
y_train_img = sat_train.iloc[:, -1]

X_test_img = sat_test.iloc[:, :-1]
y_test_img = sat_test.iloc[:, -1]

img_scaler = StandardScaler()
X_train_img = img_scaler.fit_transform(X_train_img)
X_test_img = img_scaler.transform(X_test_img)
```

### 3.2 Time-series data need ordering, lags, and time-aware validation

With time-series data, the order of the rows matters. If you shuffle the table carelessly, you may let the model learn from the future and then pretend that it generalized well.

A simple beginner method is to create lag features. For example, to predict the current CO reading, you can use the previous one, two, or three readings as extra input columns.

#### Example cell: simple lag features for a time-series task

```
air = air.sort_values("DateTime").reset_index(drop=True)

air["CO_lag_1"] = air["CO(GT)"].shift(1)
air["CO_lag_2"] = air["CO(GT)"].shift(2)
air["CO_lag_3"] = air["CO(GT)"].shift(3)

ts = air[["CO_lag_1", "CO_lag_2", "CO_lag_3", "PT08.S1(CO)", "CO(GT)"]]
ts = ts.dropna()

X_ts = ts[["CO_lag_1", "CO_lag_2", "CO_lag_3", "PT08.S1(CO)"]]
y_ts = ts["CO(GT)"]

split_index = int(len(ts) * 0.80)
X_train_ts = X_ts.iloc[:split_index]
X_test_ts = X_ts.iloc[split_index:]
y_train_ts = y_ts.iloc[:split_index]
y_test_ts = y_ts.iloc[split_index:]
```

**Important warning for forecasting**

For time-ordered problems, a random split can produce unrealistically optimistic results. TimeSeriesSplit is designed for ordered data because ordinary random folds can train on future data and test on past data [17].

## 4. Three important model families

This tutorial focuses on three model families that are easy to start with in scikit-learn: support vector machines, k-nearest neighbors, and decision trees. All three work through the same estimator interface, but they learn in very different ways [7]-[9].

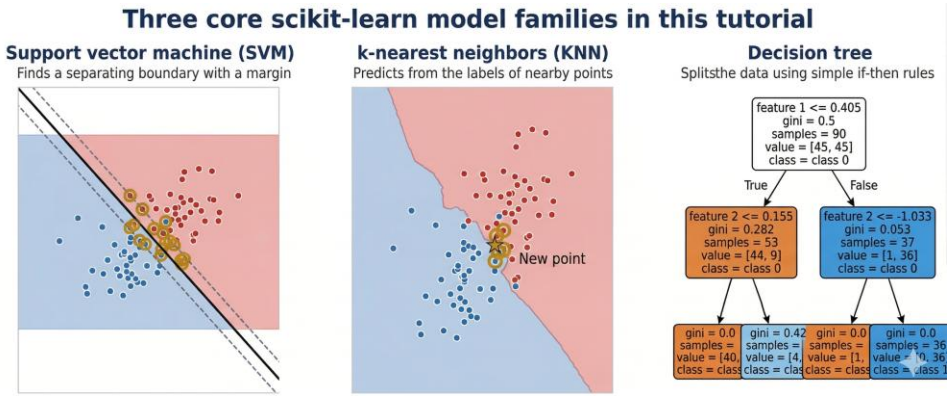



*Figure 6. Three core model families used in this tutorial: SVM, KNN, and decision trees. Author-generated visual explanation using scikit-learn classifiers and plot_tree [7]-[9].*

## 4.1 Support vector machines (SVM and SVR)

An SVM tries to find a boundary with a margin between classes. In regression, the related SVR model tries to predict a continuous value while controlling error and model complexity [7].

SVMs often work well on medium-sized numeric datasets, especially after scaling. For much larger datasets, the current scikit-learn documentation suggests considering LinearSVC or SGDClassifier instead of a full kernel SVC [7].

## 4.2 K-nearest neighbors (KNN)

KNN predicts from nearby samples in feature space. In classification it lets neighbors vote. In regression it averages nearby target values [8].

KNN is a strong teaching model because it is intuitive. However, it is very sensitive to feature scale and can slow down when the dataset becomes large [8].

## 4.3 Decision trees

A decision tree learns simple if-then rules. At each split it chooses a feature and a threshold that reduce impurity for classification or reduce error for regression [9].

Trees are easy to explain and do not require feature scaling, but very deep trees can overfit the training data. Beginners usually start by limiting max_depth or min_samples_leaf [9].

*Table 5. Focused models in this tutorial plus nearby scikit-learn alternatives students should know exist.*

| Model family | Main idea | Strengths for beginners | Watch out for | Similar scikit-learn models worth trying next |
|---|---|---|---|---|
| SVM / SVR | Use a separating boundary or regression surface with margin. | Often strong on clean numeric data; same fit/predict interface as other models. | Needs scaling; full SVC can become slow on very large data. | LinearSVC, LinearSVR, SGDClassifier, SGDRegressor |
| KNN / KNeighborsRegressor | Use nearby samples in feature space. | Very intuitive and often a good first baseline. | Sensitive to scaling and to noisy or | RadiusNeighborsClassifier, RadiusNeighborsRegressor |

| Model family | Main idea | Strengths for beginners | Watch out for | Similar scikit-learn models worth trying next |
|---|---|---|---|---|
| | | | high-dimensional data. | |
| DecisionTreeClassifier / Regressor | Split with simple if-then rules. | Easy to visualize; no scaling needed. | Can overfit if too deep. | RandomForestClassifier, RandomForestRegressor, ExtraTrees |
| Simple linear models | Fit a weighted sum of the inputs. | Excellent for baseline comparisons and interpretation. | May underfit strongly nonlinear relationships. | LinearRegression, Ridge, Lasso, LogisticRegression |
| Boosted tree ensembles | Combine many shallow trees into a stronger model. | Often powerful on tabular engineering data. | Less transparent than one small tree. | GradientBoosting, HistGradientBoosting |

### Example cell: train and visualize a small decision tree

```
from sklearn.tree import DecisionTreeRegressor, plot_tree

tree_model = DecisionTreeRegressor(max_depth=3, random_state=42)
tree_model.fit(X_train, y_train)

plt.figure(figsize=(10, 6))
plot_tree(tree_model, feature_names=X.columns, filled=True, max_depth=2)
plt.show()
```

## 5. Put scikit-learn to work in notebook cells

A clean beginner workflow is to build a pipeline, fit it on the training set, predict on the test set, and then compare a small number of alternative models. Pipelines matter because they apply the same preprocessing steps to both training and future data, which helps avoid inconsistent preprocessing and leakage [3], [6].

### Why pipelines are useful

Pipeline is one of the most important scikit-learn tools for beginners. It bundles preprocessing and the model into one object, so fit, predict, and cross-validation become simpler and safer [3], [6].

### Example cell: compare two regression models on concrete strength

```
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

svr_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVR(C=10, kernel="rbf"))
])
```

```
tree_reg = DecisionTreeRegressor(max_depth=5, random_state=42)

svr_pipe.fit(X_train, y_train)
tree_reg.fit(X_train, y_train)

svr_pred = svr_pipe.predict(X_test)
tree_pred = tree_reg.predict(X_test)

for name, pred in [("SVR", svr_pred), ("Tree", tree_pred)]:
    mae = mean_absolute_error(y_test, pred)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2 = r2_score(y_test, pred)
    print(name, "MAE=", round(mae, 2), "RMSE=", round(rmse, 2), "R2=", round(r2, 3))
```

### Example cell: compare SVC and KNN on Landsat classes

```
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

svc_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVC())
])

knn_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier(n_neighbors=5))
])

svc_model.fit(X_train_img, y_train_img)
knn_model.fit(X_train_img, y_train_img)

svc_pred = svc_model.predict(X_test_img)
knn_pred = knn_model.predict(X_test_img)

print("SVC accuracy:", round(accuracy_score(y_test_img, svc_pred), 3))
print("KNN accuracy:", round(accuracy_score(y_test_img, knn_pred), 3))
print(confusion_matrix(y_test_img, svc_pred))
```

These cells are intentionally simple. Once the basic workflow is clear, students can add parameter tuning, more advanced feature engineering, or stronger ensemble models later.

# 6. Evaluate results with the right metric

A model is only useful if it is judged with a metric that matches the engineering goal. If large errors are especially dangerous, metrics that penalize large misses more strongly may be helpful. If missed detections are costly, recall may matter more than raw accuracy [10].

*Table 6. A beginner-friendly metric guide for regression and classification [10].*

| Task | Metric | Plain-language meaning | When it is useful in civil engineering |
|---|---|---|---|
| regression | MAE | Average absolute error in the original unit. | Easy to explain when MPa, mm, or pollutant units matter directly. |

| Task | Metric | Plain-language meaning | When it is useful in civil engineering |
|---|---|---|---|
| regression | MSE / RMSE | Average squared error; RMSE returns to the original unit. | Useful when large errors should count more heavily. |
| regression | $R^2$ | How much of the target variation the model explains. | Good as a quick summary, but do not use it alone. |
| classification | accuracy | Fraction of correct predictions. | Fine when classes are balanced and mistakes have similar cost. |
| classification | precision | Of the predicted positives, how many were correct? | Useful when false alarms are expensive. |
| classification | recall | Of the actual positives, how many were found? | Useful when missing a dangerous event is expensive. |
| classification | F1-score | Single score that balances precision and recall. | Helpful when both false alarms and misses matter. |
| classification | confusion matrix | Counts of correct and incorrect predictions by class. | Best first diagnostic when you need to see what kind of mistake is happening. |

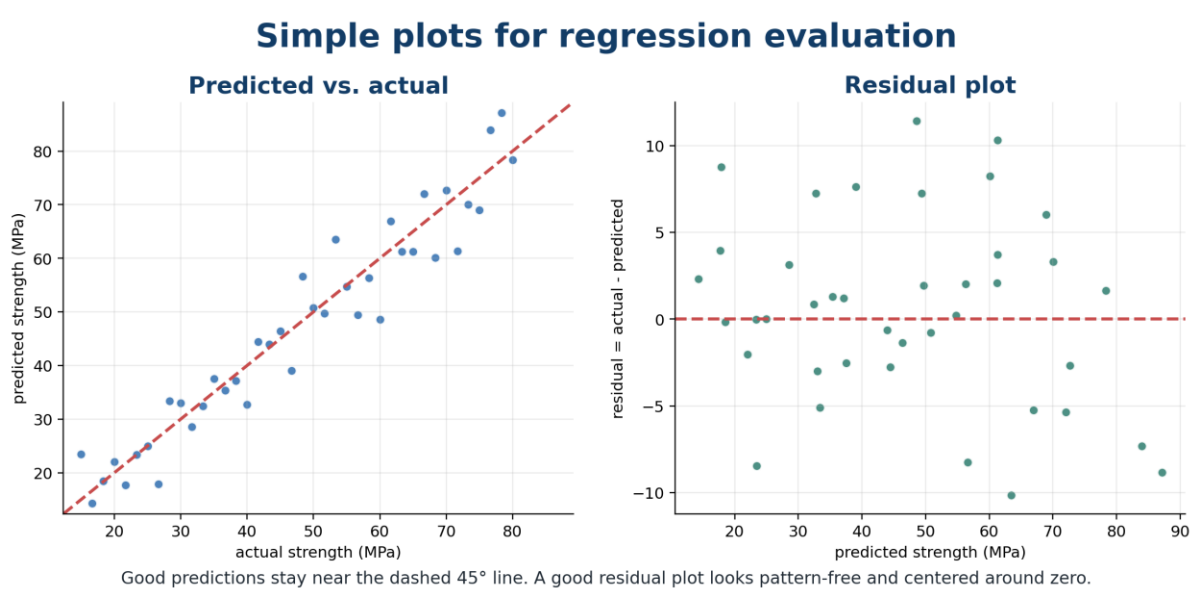



*Figure 7. Two simple plots that should become routine for regression tasks: predicted vs. actual values and a residual plot. Author-generated illustration consistent with scikit-learn's evaluation and visualization guidance [10].*

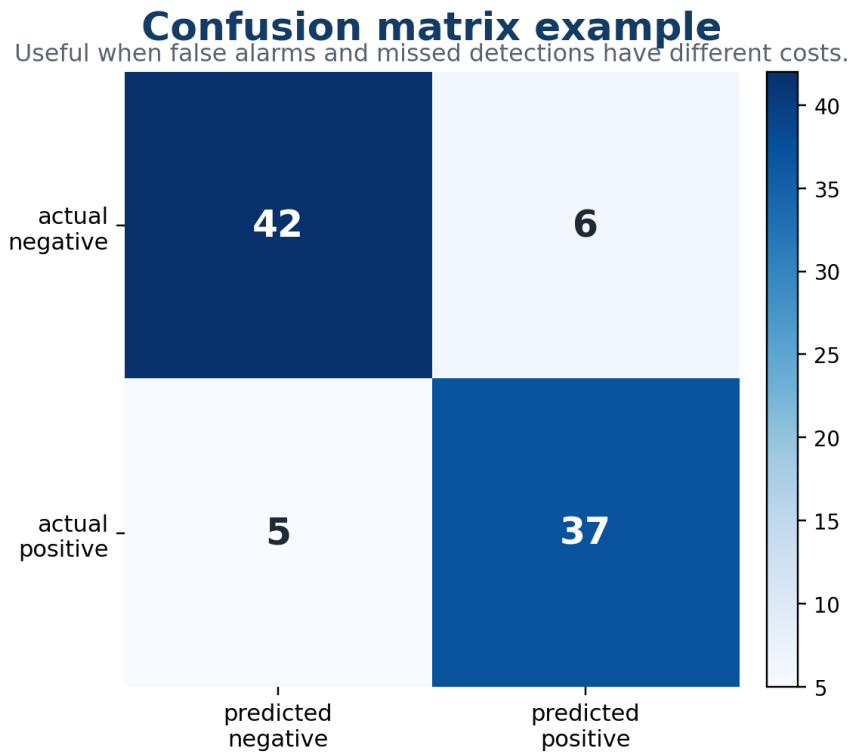



*Figure 8. A confusion matrix shows the type of classification errors, not just the overall accuracy. Author-generated illustration based on the confusion_matrix concept in scikit-learn [10].*

### Example cell: a compact metric summary

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    mean_absolute_error, mean_squared_error, r2_score
)

# regression metrics
mae = mean_absolute_error(y_test, svr_pred)
rmse = np.sqrt(mean_squared_error(y_test, svr_pred))
r2 = r2_score(y_test, svr_pred)

# classification metrics
acc = accuracy_score(y_test_img, svc_pred)
prec = precision_score(y_test_img, svc_pred, average="macro")
rec = recall_score(y_test_img, svc_pred, average="macro")
f1 = f1_score(y_test_img, svc_pred, average="macro")
```

## 6.1 Cross-validation gives a more stable estimate

One train/test split is useful, but it can be lucky or unlucky. Cross-validation repeats the train/validation process several times so you can see whether performance is stable across splits [11].

For ordinary tabular problems, KFold or StratifiedKFold are common starting choices. For time-ordered data, TimeSeriesSplit is more appropriate because it respects chronology [11], [17].

*Table 7. A simple starting guide for choosing a splitter.*

| Splitter | Good default use | Simple note |
|---|---|---|
| KFold | regression on ordinary tabular data | Shuffle if the rows have no natural order. |
| StratifiedKFold | classification with class labels | Keeps class proportions more stable across folds. |
| TimeSeriesSplit | forecasting or time-ordered sensor data | Avoids training on future values [17]. |

### Example cell: basic cross-validation for tabular and time-series data

```
from sklearn.model_selection import KFold, TimeSeriesSplit, cross_val_score

# ordinary tabular regression
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_r2 = cross_val_score(svr_pipe, X, y, cv=kf, scoring="r2")
print("mean CV R2:", round(cv_r2.mean(), 3))

# time-series example
tscv = TimeSeriesSplit(n_splits=5)
ts_mae = cross_val_score(
    tree_reg,
    X_ts,
    y_ts,
    cv=tscv,
    scoring="neg_mean_absolute_error"
)
print("mean time-series MAE:", round(-ts_mae.mean(), 3))
```

# 7. Common beginner mistakes and easy fixes

*Table 8. Very common beginner mistakes and the quickest fixes.*

| Common mistake | Why it causes trouble | Easy fix |
|---|---|---|
| scaling before the split | Information from the test set leaks into the training workflow. | Split first, then fit the scaler only on the training data, or use a Pipeline [3], [6]. |
| using accuracy only | You may hide costly false negatives or false positives. | Add precision, recall, F1, and a confusion matrix [10]. |
| randomly shuffling time-series data | The model may learn from the future. | Sort by time and use a chronological split or TimeSeriesSplit [17]. |
| growing an unrestricted tree | The tree can memorize the training data. | Start with max_depth or min_samples_leaf limits [9]. |

| Common mistake | Why it causes trouble | Easy fix |
|---|---|---|
| ignoring units and ranges | One feature can dominate because of scale, not importance. | Inspect ranges and scale when needed [4], [15], [16]. |
| dropping missing rows too early | You may throw away most of the dataset without noticing. | Count missing values first and remove rows only after checking the loss [12]. |

## 8. A simple starting path for your own project

### A sensible first project plan

1) Define the target clearly.
2) Load the table and inspect the columns.
3) Mark missing values correctly.
4) Split into train and test sets.
5) Build one small baseline pipeline.
6) Compare one or two alternative models.
7) Report metrics and plots, not just one score.
8) Write down what went wrong and what to try next.

If students follow that path, they will already understand the core scikit-learn workflow: prepare, split, fit, predict, evaluate, and improve. That is enough to begin exploring more advanced tools later, including random forests, gradient boosting, feature selection, and probability calibration, all of which are available in the same ecosystem [2], [3].

## References

[1] scikit-learn developers. Getting Started. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/getting_started.html

[2] scikit-learn developers. User Guide. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/user_guide.html

[3] scikit-learn developers. Pipelines and composite estimators. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/modules/compose.html

[4] scikit-learn developers. Preprocessing data. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/modules/preprocessing.html

[5] scikit-learn developers. train_test_split API reference. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

[6] scikit-learn developers. Common pitfalls and recommended practices. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/common_pitfalls.html

[7] scikit-learn developers. Support Vector Machines. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/modules/svm.html

[8] scikit-learn developers. Nearest Neighbors. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/modules/neighbors.html

[9] scikit-learn developers. Decision Trees. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/modules/tree.html

[10] scikit-learn developers. Metrics and scoring: quantifying the quality of predictions. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/modules/model_evaluation.html

[11] scikit-learn developers. Cross-validation: evaluating estimator performance. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/modules/cross_validation.html

[12] Vito, S. (2008). Air Quality [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C59K5F

[13] Yeh, I. (1998). Concrete Compressive Strength [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5PK67

[14] Srinivasan, A. (1993). Statlog (Landsat Satellite) [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C55887

[15] scikit-learn developers. StandardScaler API reference. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html

[16] scikit-learn developers. MinMaxScaler API reference. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html

[17] scikit-learn developers. TimeSeriesSplit API reference. scikit-learn 1.8.0 stable documentation. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html